Week 7: AI Agents

Python 200 | Mentor Session

What is an AI Agent?

A **plain LLM call**: you send a prompt, the model sends back text. One round trip.

An **AI agent**: the model decides what to do next, takes an action, observes the result, and decides again — looping until the goal is done.

The key difference: an agent can **use tools** and **act on the world**.

> **Discussion:** Think about the task "find me flights from Boston to Seattle under $400." Why can't a plain LLM call do that, but an agent might be able to?

Agents in Data Engineering

Agents are increasingly showing up in data pipelines:

- Automated ETL that adapts to unexpected data shapes
- Data quality monitoring that diagnoses and reports issues
- Code generation assistants that fix failing tests
- Pipeline orchestration that handles errors dynamically

By the end of this week you'll have built agents from scratch and with a production framework.

Tools: How Agents Act

A **tool** is a Python function the model can call.

- You define the function and describe what it does
- The model decides *when* to call it and *what arguments* to pass
- Your code runs the function — the model never executes code directly
- The result is fed back to the model

You control which tools are available. The model decides how to use them.

Tools: The Code

```python
def get_stock_price(ticker: str) -> float:
    """Returns the current stock price for the given ticker symbol."""
    # your implementation here
    return price
```

The docstring matters — it's how the model understands what the tool does.

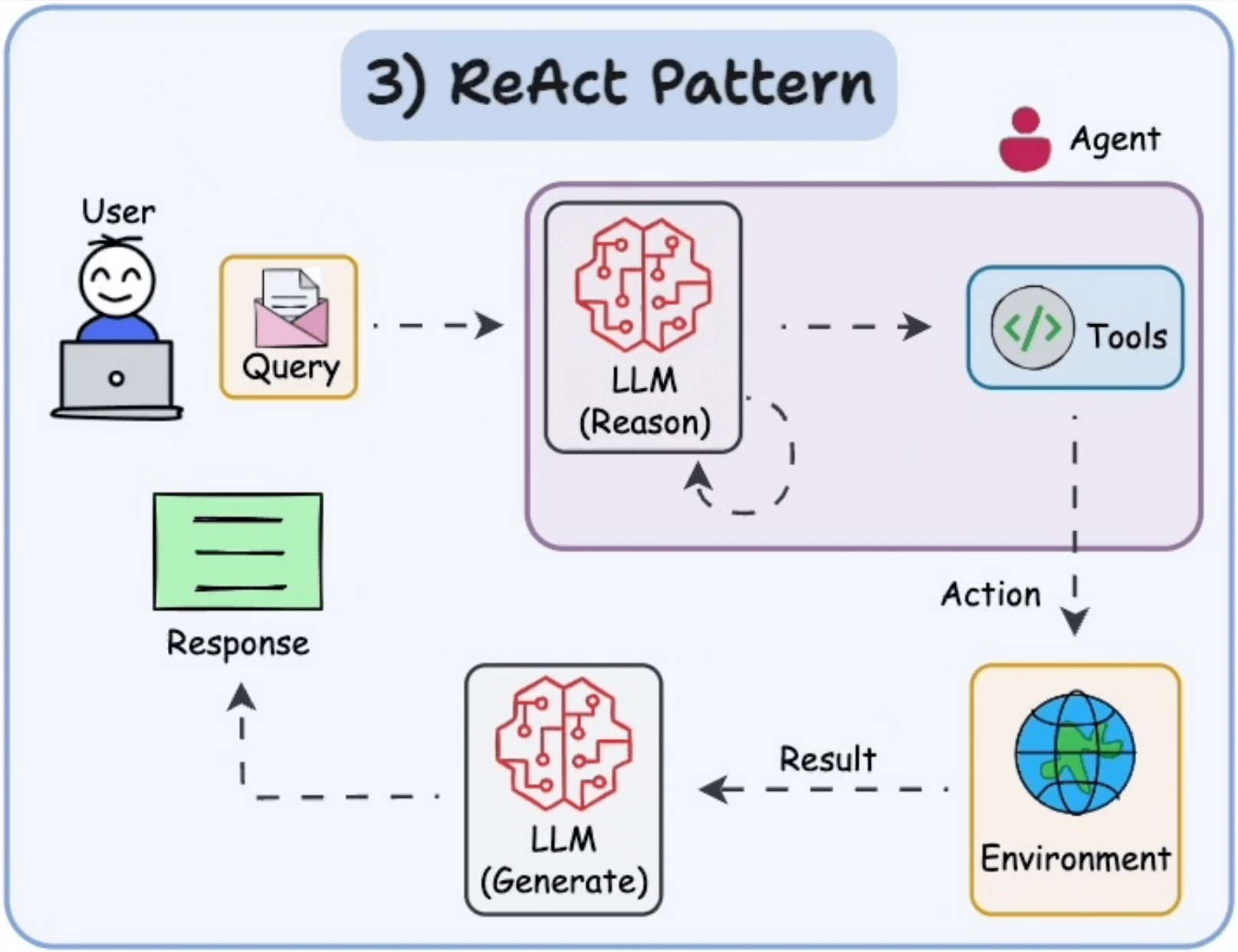
> **Discussion:** The model decides which tool to call and with what arguments, but your code actually runs the function. Why is that boundary important from a safety perspective?

The ReAct Loop

Reasoning + **Act**ing — the model interleaves thinking and doing:

1. **Reason** — look at the goal and current state, decide what to do next
2. **Act** — call a tool (run a function, query an API, search the web)
3. **Observe** — read the result of the tool call
4. **Repeat** — reason again with the new information, until the goal is met

See next slide for diagram →


3) ReAct Pattern
Agent
User
Query
LLM
(Reason)
Tools
Action
Response
LLM
(Generate)
Result
Environment

Kaggle Demo: Hello, Agent *(Lesson 2)*

Open: Lesson 2 → "Open in Kaggle" button

Focus on	Skim for now
How a tool is defined and described to the model	Low-level OpenAI tool spec JSON
How the model decides to call the tool	Multi-tool routing
The full loop: prompt → tool call → result → answer	Error handling patterns

This is the "Hello World" of agents — one tool, one loop. Get comfortable with the mechanics first.

> **Discussion:** Walk through the ReAct loop for this demo. Where does the model reason? Where does it act? Where does it observe?

Building an ETL Agent

Instead of a hardcoded pipeline script, define tools for each step:

- `extract_data()` — pull from the source
- `transform_data()` — clean and reshape
- `load_to_db()` — write to the destination

Then give the agent a goal: *"Process today's sales data."*

The model decides the order, calls each tool, and handles unexpected results.

LLM as Orchestrator

This is a pattern you'll see in production:

LLM as orchestrator → tools as workers

The model coordinates. The tools do the actual work. Your code defines what's possible; the agent decides what to do.

> **Discussion:** Compare this to a traditional ETL script where every step is hardcoded. What's an advantage of the agent approach? What's a risk?

Kaggle Demo: ETL Agent *(Lesson 3)*

Open: Lesson 3 → "Open in Kaggle" button

Watch for	Why it matters
The model calling tools in sequence	It's deciding the order, not just executing
What happens when a tool returns an error	The model can reason about failure and adapt

Intentionally break one of the tools — return a wrong result or raise an exception. Watch how the agent responds.

> **Discussion:** The agent handles some surprises, but not all. What kinds of failures would you want to catch in your code before the agent even tries?

smolagents: A Production Framework

Building an agent from scratch took a lot of code — loop management, tool routing, error handling.

smolagents (HuggingFace) handles all of that:

```
from smolagents import CodeAgent, tool

@tool
def get_weather(city: str) -> str:
    """Returns current weather for the given city."""
    ...

agent = CodeAgent(tools=[get_weather], model=model)
agent.run("What's the weather in Seattle?")
```

The `@tool` decorator is all you need to expose a function to the agent.

Tool-based vs. Code-based Agents

Tool-based	Code-based
Model calls pre-defined functions	Model writes and executes arbitrary code
Safer — you control what's available	More powerful — solves novel problems
Easier to audit and reason about	Requires a sandbox; harder to predict

Most production pipelines use tool-based agents. Code-based agents are powerful but need careful guardrails.

> **Discussion:** A code-based agent can write Python and run it to solve problems you didn't anticipate when building the tool list. What's exciting about that? What's concerning?

Kaggle Demo: smolagents *(Lesson 4)*

Open: Lesson 4 → "Open in Kaggle" button

Focus on	Skim for now
The `@tool` decorator and how easy tool definition becomes	Internals of the ReAct loop
Comparing smolagents code to the DIY version	Multi-agent pipelines
Running a tool-based vs. code-based agent	

> **Discussion:** What would you need to trust before deploying an agent that could write and run code in a production environment?

AI Code Assistants *(Lesson 5 Demo)*

The lesson demo uses an AI code agent to:

- Evaluate an existing project
- Fix failing tests
- Generate documentation

Tools you'll encounter in the wild:

- **GitHub Copilot** — inline autocomplete, function generation
- **Claude Code** — multi-file context, refactoring, explaining unfamiliar codebases

Where Code Agents Shine (and Don't)

Good at	Not so good at
Boilerplate and repetitive patterns	Novel architecture decisions
Reading and summarizing existing code	Debugging subtle runtime behavior
Writing tests for known logic	Knowing when to stop

> **Discussion:** Have you used an AI coding assistant? What was the most useful thing it did? What did it confidently get wrong?

Agents: When to Use Them

Use an agent when:

- The task requires multiple steps and the order isn't fixed
- The pipeline needs to adapt to unexpected results
- You want the model to orchestrate across tools

Use a regular script when:

- The steps are fixed and well-understood
- Predictability and auditability matter more than flexibility
- Cost and latency are a concern

> **Discussion:** You're building a data pipeline that runs every night to process sales data. Agent or script? Why?

Week 7 Wrap-Up

- **AI agent** — an LLM that can use tools and loop until the goal is done
- **ReAct loop** — Reason → Act → Observe, repeat; the core pattern
- **Tools** — functions you define; the model decides when and how to call them
- **ETL agent** — LLM as orchestrator, tools as workers
- **smolagents** — production framework: less code, better defaults
- **Tool-based vs. code-based** — control vs. flexibility; both have their place

Takeaway: Agents are showing up in data engineering pipelines. Understanding the pattern means you can evaluate when to use one — and when a simpler script is the right call.